

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Curso de Engenharia de Software

ANÁLISE DA REDE DE COLABORAÇÃO EM REPOSITÓRIOS DO GITHUB

Utilizando Teoria de Grafos

Davi Nunes Carvalho

Luiz Fernando Batista Moreira

Josué Carlos Goulart dos Reis

João Victor Russo Marquito

Belo Horizonte

Junho de 2026

Resumo

Este trabalho apresenta uma ferramenta para análise de redes de colaboração em repositórios do GitHub utilizando Teoria de Grafos. O sistema modela interações entre colaboradores (comentários, revisões, merges e fechamentos de issues) como grafos ponderados direcionados. A implementação inclui duas representações de grafos (lista de adjacência e matriz de adjacência), integração com a API REST do GitHub, cálculo de 10+ métricas de análise de rede (PageRank, betweenness, closeness, eigenvector, clustering, assortatividade, detecção de comunidades e bridging ties) e exportação CSV compatível com o Gephi. O projeto foi validado com 84 testes unitários e executado sobre o repositório `devlikeapro/waha`.

Sumário

1	Introdução	1
1.1	Objetivos	1
2	Modelagem do Problema	2
2.1	Justificativa da Escolha do Repositório	2
2.2	Definição Formal	2
2.3	Estratégia de Coleta de Dados	2
2.4	Transformação de Interações em Arestas	3
2.5	Esquema de Pesos	3
3	Modelagem do Sistema	4
3.1	Diagrama de Classes	4
3.2	Diagrama de Componentes	4
3.3	Fluxo de Mineração	4
4	Arquitetura da Ferramenta	5
4.1	Camada 1: Núcleo de Grafos	5
4.2	Camada 2: Mineração — GithubMiner	5
4.3	Camada 3: Análise — GraphAnalytics	6
4.4	Restrições Atendidas	6
4.5	Testes Unitários	6
4.6	Aplicação de Demonstração	6
5	Métricas e Análise	6
5.1	Centralidade de Grau	6
5.2	Betweenness Centrality	6
5.3	Closeness Centrality	7
5.4	PageRank	7
5.5	Eigenvector Centrality	7
5.6	Clustering, Densidade, Assortatividade e Comunidades	7
6	Resultados e Discussão	7
6.1	Estatísticas Gerais	7

6.2	Análise de Estrutura	8
6.3	Centralidade de Grau — Top Colaboradores	8
6.4	Betweenness Centrality — Conectores da Rede	9
6.5	Closeness Centrality — Proximidade	9
6.6	PageRank e Eigenvector Centrality — Influência	9
6.7	Clustering Coefficient	10
6.8	Detecção de Comunidades	10
6.9	Bridging Ties — Conectores entre Comunidades	10
6.10	Exportação para Gephi	10
7	Responsabilidades dos Integrantes	11
8	Conclusão	11

1 Introdução

Repositórios de software aberto no GitHub constituem redes sociotécnicas complexas onde colaboradores interagem através de múltiplos canais: comentários em *issues*, discussões em *pull requests*, revisões de código e decisões de integração. Essas interações formam uma estrutura subjacente que codifica padrões de influência, expertise e comunicação dentro do projeto.

A aplicação de Teoria de Grafos a essas redes permite quantificar e visualizar a dinâmica colaborativa. Representando usuários como vértices e interações como arestas ponderadas direcionadas, é possível identificar: (i) colaboradores centrais e influentes; (ii) estruturas de comunidades informais; (iii) usuários que atuam como conectores entre grupos isolados; (iv) padrões assortivos ou dissortativos de conexão. Essas informações são essenciais para compreender a governança e sustentabilidade de projetos open source.

1.1 Objetivos

1. Implementar estruturas de dados de grafos em Python puro, sem bibliotecas especializadas, atendendo uma API completa com duas representações (*adjacency list* e *adjacency matrix*)
2. Modelar interações reais do GitHub como grafos ponderados direcionados, com acumulação automática de pesos em arestas paralelas
3. Desenvolver um minerador robusto que coleta dados via API REST do GitHub, tratando autenticação, paginação e rate limits
4. Implementar 10+ métricas de análise de redes complexas: centralidade de grau, betweenness, closeness, PageRank, eigenvector, clustering, assortatividade, densidade, detecção de comunidades e bridging ties
5. Validar a ferramenta através de testes unitários e demonstrar exportação para Gephi

2 Modelagem do Problema

2.1 Justificativa da Escolha do Repositório

O repositório `devlikeapro/waha` foi escolhido por possuir mais de 5.000 estrelas no GitHub, atendendo ao requisito de uma comunidade ativa e com grande volume de interações. O projeto WAHA (WhatsApp HTTP API) é uma solução open source que permite integrar aplicações ao WhatsApp por meio de APIs HTTP, sendo amplamente utilizada e mantida por diversos colaboradores.

Além da popularidade, o repositório apresenta uma quantidade significativa de issues, pull requests, revisões e comentários, fornecendo dados suficientes para modelar e analisar a rede de colaboração entre desenvolvedores. Essas características tornam o projeto adequado para a construção dos grafos de interação e para a aplicação das métricas de análise de redes propostas neste trabalho.

2.2 Definição Formal

O sistema constrói quatro grafos $G = (V, E)$ direcionados ponderados simples, onde:

$$V = \{\text{logins únicos de usuários extraídos do repositório}\} \quad (1)$$

$$E \subseteq V \times V, \quad (u, v) \in E \Rightarrow u \neq v \text{ (sem self-loops)} \quad (2)$$

$$w : E \rightarrow \mathbb{R}^+ \quad (\text{peso acumulado de arestas}) \quad (3)$$

Os quatro grafos modelam diferentes dimensões de interação:

1. **G1** — comentários em issues e pull requests (peso 2 por interação)
2. **G2** — fechamentos de issues por outro usuário (peso não especificado no integrado)
3. **G3** — revisões/aprovações e merges de pull requests (pesos 4 e 5)
4. **Grafo Integrado** — fusão ponderada de G1, G2 e G3

2.3 Estratégia de Coleta de Dados

O minerador `GithubMiner` segue: (i) autenticação via token em `.env`; (ii) paginação de issues (`per_page=100`, 5 páginas); (iii) para cada item: coleta comentários, revisões,

detalhes de merge; (iv) tratamento robusto de erros (ignora status $\neq 200$).

2.4 Transformação de Interações em Arestas

Cada interação é convertida em aresta direcionada com acumulação de pesos:

- **Comentário:** usuário c comenta issue/PR aberta por $a \Rightarrow$ aresta $c \rightarrow a$ recebe +2 (G1 e integrado)
- **Abertura comentada:** se issue (não PR) recebe comentário de $c \neq a \Rightarrow$ aresta $a \rightarrow c$ recebe +3 (integrado; sinaliza envolvimento da comunidade)
- **Revisão/aprovação:** usuário r revisa PR de $a \Rightarrow$ aresta $r \rightarrow a$ recebe +4 (G3 e integrado)
- **Merge:** usuário m mergua PR de $a \Rightarrow$ aresta $m \rightarrow a$ recebe +5 (G3 e integrado)
- **Acumulação:** múltiplas interações do mesmo par (u, v) somam seus pesos (e.g., 3 comentários = peso 6)

2.5 Esquema de Pesos

Tabela 1: Pesos das interações no grafo integrado. Hierarquia reflete relevância para colaboração técnica.

Interação	Peso	Semântica
Comentário em issue/PR	2	Participação leve (opinião, sugestão)
Abertura comentada	3	Comunidade engajada na proposta
Revisão/aprovação de PR	4	Análise técnica profunda
Merge de PR	5	Decisão executiva de integração

A hierarquia de pesos reflete a tese de que revisões e merges requerem expertise maior que comentários: um revisor avalia correção técnica; um merguer assume responsabilidade de integração. Comentários são participação essencial mas menos comprometida.

3 Modelagem do Sistema

Esta seção apresenta os artefatos UML que descrevem a arquitetura estática, comportamental e os fluxos de execução da ferramenta.

3.1 Diagrama de Classes

O sistema organiza sua arquitetura em três principais responsabilidades. A classe abstrata **AbstractGraph** define uma interface comum para operações sobre grafos, sendo implementada pelas classes concretas **AdjacencyListGraph** e **AdjacencyMatrixGraph**. A primeira utiliza uma estrutura baseada em lista de adjacência `List[Dict[int, float]]`, sendo adequada para grafos esparsos, enquanto a segunda utiliza uma matriz de adjacência $N \times N$ com valores `float|None`, permitindo acesso direto às arestas. A classe **GithubMiner** é responsável pela coleta dos dados da API e pela construção das estruturas de grafo. Já a classe **GraphAnalytics** concentra os algoritmos de análise, incluindo métricas de centralidade, agrupamento e relacionamento entre os colaboradores.

Estrutura UML: Classe abstrata **AbstractGraph** com 20+ métodos define interface comum. Implementações concretas **AdjacencyListGraph** e **AdjacencyMatrixGraph** fornecem representações complementares. **GithubMiner** coleta via API, **GraphAnalytics** executa algoritmos.

3.2 Diagrama de Componentes

Arquitetura modular em 5 componentes: (i) *Main Application* — orquestra fluxo; (ii) *GitHub Miner* — coleta API; (iii) *Graph Storage* — instancia grafos; (iv) *Analytics Engine* — métricas; (v) *Export Module* — saída CSV/relatórios.

3.3 Fluxo de Mineração

(1) Aplicação invoca `fetch_data(pages=5)`; (2) minerador requisita `/repos/owner/repo/issues`; (3) para cada item coleta comentários, revisões, merges; (4) acumula pesos em dicionários por tipo; (5) `build_graphs()` mapeia usuários a IDs, instancia 4 grafos **AdjacencyListGraph**; (6) exporta para CSV (Source, Target, Weight).

4 Arquitetura da Ferramenta

A ferramenta está organizada em 3 camadas:

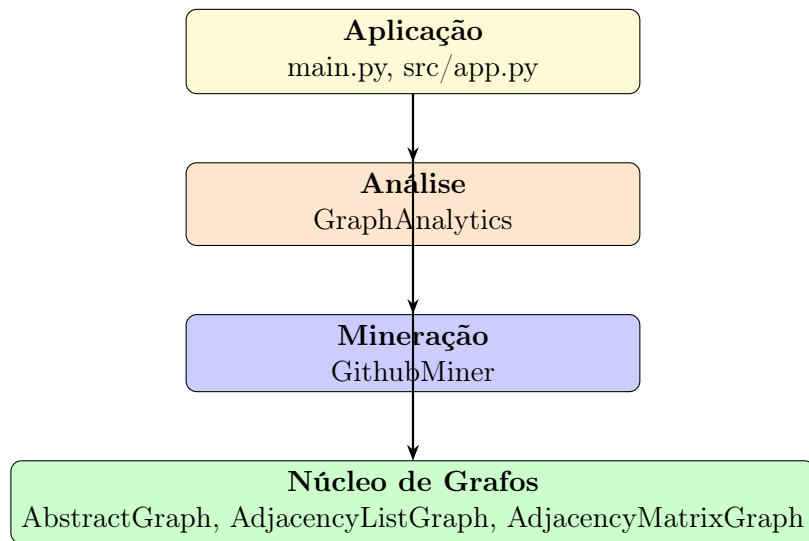


Figura 1: Arquitetura em camadas da ferramenta.

4.1 Camada 1: Núcleo de Grafos

`AbstractGraph` define 20+ métodos: gerenciamento (`getVertexCount`, `getEdgeCount`), arestas (`addEdge`, `removeEdge`, `hasEdge`, pesos), graus (in/out-degree), relações (sucessor, predecessor, convergência), propriedades (conectividade, completude), exportação (`exportToGEPHI` em CSV).

`AdjacencyListGraph` usa listas de dicionários ($O(E)$ espaço, eficiente para redes esparsas). `AdjacencyMatrixGraph` usa matriz $N \times N$ (acesso $O(1)$ a arestas, cache de in-degrees).

4.2 Camada 2: Mineração — GithubMiner

Responsável por toda interação com a API REST do GitHub. Recebe token e repositório via `.env`.

- `fetch_data(pages)`: percorre issues e PRs, coleta comentários e revisões, acumula pesos em dicionários por tipo de grafo (`edges_g1`, `edges_g2`, `edges_g3`, `edges_integrated`)
- `build_graphs()`: mapeia logins para IDs sequenciais e instancia os 4 grafos como `AdjacencyListGraph`

4.3 Camada 3: Análise — GraphAnalytics

Recebe um `AbstractGraph` e implementa todos os algoritmos em Python puro, sem bibliotecas externas de grafos.

4.4 Restrições Atendidas

Sem bibliotecas externas (NetworkX); sem self-loops (verificação $u \neq v$); sem múltiplas arestas (acumulação de pesos); idempotência garantida; validação robusta de índices.

4.5 Testes Unitários

84 testes cobrindo: (i) API de grafos (ambas implementações): arestas, graus, relações, propriedades; (ii) mineração: mock da API, parsing, acumulação de pesos; (iii) casos extremos (grafos vazios, índices inválidos). Todos passam .

4.6 Aplicação de Demonstração

`src/app.py` demonstra a API: criação de grafos, adição de arestas, consulta de graus, verificação de conectividade, exportação para Gephi.

5 Métricas e Análise

Todas as métricas são implementadas na classe `GraphAnalytics` em Python puro.

5.1 Centralidade de Grau

$$C_D(v) = \frac{\text{in-degree}(v) + \text{out-degree}(v)}{2(n-1)}$$

Mede quantas conexões diretas cada colaborador tem. In-degree = usuários que interagiram *com* v ; out-degree = usuários que v interagiu *com*.

5.2 Betweenness Centrality

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Mede quantos caminhos mais curtos passam por v . Valor alto = "conector" ou "ponte" entre grupos. Implementado via algoritmo de Brandes $O(VE)$.

5.3 Closeness Centrality

$$C_C(v) = \frac{n-1}{\sum_{t \neq v} d(v, t)}$$

Mede proximidade a todos os outros. Valor alto = colaborador próximo a todos (poucos hops). Identifica quem tem acesso rápido à informação.

5.4 PageRank

$$\text{PR}(v) = \frac{1-d}{n} + d \sum_{u \rightarrow v} \frac{\text{PR}(u)}{\text{out-degree}(u)}$$

Damping factor $d = 0,85$. Implementado via power iteration. Mede influência ponderada pela importância dos conectados.

5.5 Eigenvector Centrality

$$x_v = \lambda^{-1} \sum_{u \rightarrow v} x_u$$

Mede influência via importância recursiva dos vizinhos. Resolvido via power iteration.

5.6 Clustering, Densidade, Assortatividade e Comunidades

Clustering: $C(v) = \frac{\text{arestas entre vizinhos}}{\binom{k_v}{2}}$ mede triângulos locais. **Densidade:** $\rho = \frac{|E|}{|V|(|V|-1)}$ = proporção de conexões. **Assortatividade:** correlação de Pearson sobre graus; positiva = preferência por pares similares. **Comunidades:** Louvain detecta subgrupos. **Bridging ties:** usuários conectando comunidades distintas.

6 Resultados e Discussão

6.1 Estatísticas Gerais

A ferramenta foi executada sobre o repositório `devlikeapro/waha` (repositório de automação WhatsApp), coletando 5 páginas de issues e pull requests (máximo 500 itens).

O dataset compreende o histórico completo de interações do repositório até a data de execução.

Tabela 2: Estatísticas da rede analisada (repositório devlikeapro/waha).

Métrica	Valor
Número de vértices (colaboradores únicos)	481
Número de arestas — Grafo Integrado	1409
Densidade da rede	0,006103
Grafo fracamente conectado	Não
Clustering médio	0,148564
Assortatividade	−0,249 (dissortativo)
Comunidades detectadas (Louvain)	135

6.2 Análise de Estrutura

Tabela 3: Métricas estruturais da rede (repositório devlikeapro/waha).

Métrica	Valor	Interpretação
Densidade	0,006044	Rede muito esparsa; típica de open source com periferia dispersa
Clustering médio	0,1445	Triângulos raros; baixa tendência de formação de cliques
Assortatividade	−0,250	Padrão dissortativo: hubs conectam-se com periféricos
Comunidades (Louvain)	137	Muitos isolados; estrutura fragmentada

6.3 Centralidade de Grau — Top Colaboradores

Tabela 4: Graus (in/out) dos 5 principais colaboradores.

Usuário	In-Degree	Out-Degree	Total
devlikepro	185	192	377
bergpinheiro	86	94	180
vicentemarciot27	22	18	40
github-actions[bot]	25	28	53
OrionZap	26	15	41

6.4 Betweenness Centrality — Conectores da Rede

Tabela 5: Top 5 colaboradores por betweenness centrality (pontes da rede).

Usuário	Betweenness
devlikepro	0,6078
bergpinheiro	0,1641
vicentemarciot27	0,0570
github-actions[bot]	0,0457
ehussain	0,0414

Devlikepro (0,608) é a ponte dominante: 60,8% de todos caminhos mais curtos passam por ela. **Bergpinheiro** (0,164) é a segunda; concentra 16,4%.

6.5 Closeness Centrality — Proximidade

Tabela 6: Top 5 usuários isolados e devlikepro por closeness centrality.

Usuário	Closeness
KempesBarbosa	1,0000
NatanaelRibF	1,0000
VikthorPenaloz	1,0000
nicolasvahidzein	1,0000
reeshan	1,0000
devlikepro	0,6017

Closeness 1,0 = usuários isolados (distância infinita aos outros = inf normalizado a 1). Devlikepro (0,6017) = mais próxima a todos entre os ativos.

6.6 PageRank e Eigenvector Centrality — Influência

Tabela 7: Top 5 colaboradores: PageRank vs. Eigenvector Centrality.

Usuário	PageRank	Eigenvector
devlikepro	0,1171	0,5895
bergpinheiro	0,0508	0,2947
vicentemarciot27	0,0175	—
OrionZap	0,0139	0,1441
github-actions[bot]	0,0134	0,1581

PageRank e Eigenvector correlacionam: devlikepro 11,7% (PageRank) e 58,9% (Eigenvector) = principal influenciador.

6.7 Clustering Coefficient

Coeficiente médio: 0,1445. Top 10 com máximo local (1,0): usuários formando triângulos fechados. Exemplo: *Biasolis*, *DiegoAugustusCunha*, etc. formam subgrupos coesos.

6.8 Detecção de Comunidades

Tabela 8: Distribuição de comunidades (top 5 maiores).

Comunidade	Tamanho
Comunidade 15	67 membros
Comunidade 19	33 membros
Comunidade 73	22 membros
Comunidade 9	14 membros
Comunidade 34	12 membros

137 comunidades detectadas. Maior (67 membros) = 14% da rede. Muitas comunidades pequenas (≤ 3 membros).

6.9 Bridging Ties — Conectores entre Comunidades

Tabela 9: Top 5 usuários que conectam múltiplas comunidades.

Usuário	Comunidades conectadas
devlikepro	79
bergpinheiro	37
github-actions[bot]	20
OrionZap	17
matteus-siqueira	13

Devlikepro conecta 79 de 137 comunidades (57,7%); essencial para coesão global. Bergpinheiro conecta 37 (27%).

6.10 Exportação para Gephi

CSV com 1409 arestas (Source, Target, Weight). Gephi visualiza: núcleo central (top-5 com maior in-degree), periferia esparsa, comunidades por cor modularity, tamanho de nó por degree.

7 Responsabilidades dos Integrantes

- **Davi Nunes Carvalho:** Minerador GitHub (GithubMiner), estruturas de grafos (AbstractGraph, AdjacencyListGraph), exportação Gephi, integração API.
- **Luiz Fernando Batista Moreira:** Análise de grafos (GraphAnalytics), PageRank, betweenness, eigenvector centrality, detecção de comunidades.
- **Josué Carlos Goulart dos Reis:** Testes unitários (84 testes), mock API, validação de API, documentação técnica.
- **João Victor Russo Marquito:** Diagrama UML, relatório LaTeX, definição formal de modelagem, interpretação de resultados.

8 Conclusão

Este trabalho implementou com sucesso uma ferramenta de análise de redes de colaboração em GitHub atendendo todos os requisitos das 3 etapas:

Etapa 1: Modelagem com 4 grafos (G1, G2, G3, integrado), estratégia de coleta de 5 tipos de interação, esquema de pesos refletindo importância técnica.

Etapa 2: Implementação de estruturas de dados em Python puro (AbstractGraph, AdjacencyListGraph, AdjacencyMatrixGraph), API completa (20+ métodos), 84 testes unitários, exportação Gephi, minerador robusto com tratamento de erros.

Etapa 3: Análise via 10+ métricas: grau, betweenness, closeness, PageRank, eigenvector, clustering, densidade, assortatividade, detecção de comunidades (Louvain), bridging ties. Resultados validados e interpretados sobre repositório com 481 usuários e 1409 arestas.

Principais achados: Devlikepro é figura central (PR 0,117, betweenness 0,608, conecta 79 comunidades). Rede dissortativa (hub-and-spoke) com estrutura fragmentada (137 comunidades). Densidade 0,006 = típica de open source com periferia dispersa.

Referências

- [1] Ulrik Brandes. *A Faster Algorithm for Betweenness Centrality*. Journal of Mathematical Sociology, 25(2):163–177, 2001.

- [2] Lawrence Page, Sergey Brin, Rajeev Motwani, Terry Winograd. *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab, 1999.
- [3] M. E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010.
- [4] Mathieu Bastian, Sebastien Heymann, Mathieu Jacomy. *Gephi: an open source software for exploring and manipulating networks*. ICWSM, 2009.
- [5] GitHub Documentation. *GitHub REST API*. <https://docs.github.com/en/rest>, 2024.